

TRAX ILIAS BRIDGE

Documentation technique détaillée de l'adaptateur

Installation, configuration, exploitation et diagnostic - Trax ILIAS Bridge v2.0.3

Version documentée	2.0.3
Date	16/06/2026
Audience	Administrateurs systèmes, administrateurs Trax, intégrateurs ILIAS
Périmètre	ILIAS 10 + Trax LRS 3 - cmi5, Learning Experiences, Ranking
Fichiers étudiés	aggregate.php, apache-trax-ilias-adapter.conf, config.sample.php

Document rédigé à partir de l'archive source v2.0.3 fournie et du comportement validé : cmi5 OK, Learning Experiences OK, Ranking OK, xAPI non-cmi5 non perturbé.

Table des matières

- 1. Architecture technique**
- 2. Composants livrés**
- 3. Prérequis**
- 4. Installation**
- 5. Configuration Apache**
- 6. Configuration applicative**
- 7. Flux HTTP**
- 8. Authentification et en-têtes**
- 9. Tests techniques**
- 10. Supervision et logs**
- 11. Performance**
- 12. Sécurité**
- 13. Dépannage**
- 14. Upgrade et rollback**
- 15. Annexes techniques**

Les titres du document utilisent les styles Word afin de pouvoir générer ou mettre à jour une table des matières automatique dans Word si nécessaire.

1. Architecture technique

L'adaptateur est installé sur le serveur Trax, devant l'application Trax/Laravel. Apache intercepte uniquement des routes précises et les redirige vers le script PHP aggregate.php. Les autres routes Trax, y compris /xapi/statements, continuent vers Trax sans intervention de l'adaptateur.

```

ILIAS 10
|
| Endpoint LRS configuré : /trax/api/gateway/clients/{client}/stores/{store}/xapi
v
Apache serveur Trax
|-- AliasMatch /cmi5/tokens          -> aggregate.php -> Trax réel
|-- AliasMatch /api/statements/aggregate -> aggregate.php -> Trax /xapi/statements
|-- /xapi/statements                 -> Trax réel directement
v
Trax LRS 3

```

Principe de sécurité fonctionnelle

La configuration v2.0.3 est volontairement restrictive : elle n'intercepte pas toute l'API Trax, mais seulement trois URL nécessaires à la compatibilité ILIAS.

2. Composants livrés

Fichier	Emplacement cible recommandé	Rôle
aggregate.php	/var/www/trax-ili-as-aggregate-adapter/aggregate.php	Contrôleur unique : proxy cmi5 tokens et endpoint de compatibilité aggregate.
config.sample.php	/var/www/trax-ili-as-aggregate-adapter/config.php	Modèle de configuration applicative PHP.
apache-trax-ili-as-adapter.conf	/etc/httpd/conf.d/trax-ili-as-aggregate-adapter.conf	Configuration Apache AliasMatch.
README.md	Dépôt Git	Documentation courte d'installation et historique de version.

3. Prérequis

- Serveur Trax LRS 3 fonctionnel, accessible en HTTP ou HTTPS.
- Apache HTTPD avec PHP activé sur le serveur Trax.
- PHP 8.1 minimum recommandé par le code utilisé, PHP 8.2 conseillé si l'environnement Trax l'utilise déjà.
- Extension PHP cURL active.
- Accès administrateur pour déployer sous /var/www et /etc/httpd/conf.d.
- Un client Trax et un store Trax disponibles pour ILIAS.
- ILIAS 10 configuré avec un type LRS xAPI/cmi5 pointant vers /xapi.

```

php -v
php -m | grep -i curl
apachectl -M | grep -E "alias|php|proxy|rewrite"

```

4. Installation

4.1 Déploiement des fichiers

```

mkdir -p /var/www/trax-ili-as-aggregate-adapter

cp aggregate.php /var/www/trax-ili-as-aggregate-adapter/aggregate.php
cp config.sample.php /var/www/trax-ili-as-aggregate-adapter/config.php

```

```
chown -R apache:apache /var/www/trax-ili-as-aggregate-adapter
chmod 0644 /var/www/trax-ili-as-aggregate-adapter/aggregate.php
chmod 0640 /var/www/trax-ili-as-aggregate-adapter/config.php
```

4.2 Déploiement Apache

```
cp apache-trax-ili-as-adapter.conf /etc/httpd/conf.d/trax-ili-as-aggregate-adapter.conf

apachectl configtest
systemctl restart httpd
systemctl status httpd --no-pager
```

4.3 Vérification du chargement

```
grep -nE "AliasMatch|cmi5/tokens|statements/aggregate" /etc/httpd/conf.d/trax-ili-as-aggregate-adapter.conf
apachectl -S
```

5. Configuration Apache

La version 2.0.3 utilise AliasMatch plutôt que RewriteRule. Ce choix force l'interception au niveau Apache pour les installations où les règles de réécriture en contexte serveur ne passent pas avant le routeur Laravel de Trax.

Ligne fonctionnelle	Expression	Rôle
Learning Experiences / Ranking sans store	<code>^/trax/api/gateway/clients/[^]+/stores/api/statements/aggregate\$</code>	Corrige le cas où ILIAS construit une URL aggregate en perdant le nom du store.
Learning Experiences / Ranking avec store	<code>^/trax/api/gateway/clients/[^]+/stores/[^]+/api/statements/aggregate\$</code>	Supporte les appels aggregate qui contiennent déjà le store.
Pré-lancement cmi5	<code>^/trax/api/gateway/clients/[^]+/stores/[^]+/cmi5/tokens\$</code>	Nettoie activity_id avant transmission à Trax.

Fichier Apache complet

```
# Trax ILIAS Bridge - Apache configuration
#
# Version 2.0.3
# Configuration volontairement basée sur AliasMatch : dans certaines installations
# Apache/Trax, les RewriteRule en contexte serveur ne sont pas appliquées avant
# Laravel. AliasMatch force l'interception par l'adaptateur.
#
# Noms conservés par rapport à la première version :
# - apache-trax-ili-as-adapter.conf
# - aggregate.php
# - /var/www/trax-ili-as-aggregate-adapter

# 1) Learning Experiences / Ranking avec URL native ILIAS 10 qui perd le store :
# /clients/{client}/stores/api/statements/aggregate
AliasMatch "^/trax/api/gateway/clients/[^]+/stores/api/statements/aggregate$" "/var/www/trax-ili-as-aggregate-adapter/aggregate.php"

# 2) Learning Experiences / Ranking avec URL contenant le store :
# /clients/{client}/stores/{store}/api/statements/aggregate
AliasMatch "^/trax/api/gateway/clients/[^]+/stores/[^]+/api/statements/aggregate$" "/var/www/trax-ili-as-aggregate-adapter/aggregate.php"

# 3) Pré-lancement cmi5 :
# ILIAS 10 envoie activity_id ; l'adaptateur le supprime puis transmet à Trax.
AliasMatch "^/trax/api/gateway/clients/[^]+/stores/[^]+/cmi5/tokens$" "/var/www/trax-ili-as-aggregate-
```

```

adapter/aggregate.php"

<Directory "/var/www/trax-ili-as-aggregate-adapter">
    Require all granted
    Options -Indexes
    AllowOverride None
</Directory>

# Permet à PHP de récupérer l'en-tête Authorization si nécessaire.
SetEnvIf Authorization "(.*)" HTTP_AUTHORIZATION=$1

```

Contrôle important

Il ne doit pas y avoir de règle globale du type AliasMatch ^/trax/api/gateway/clients/.* qui capturerait /xapi/statements. Une telle règle perturberait les traces xAPI non-cmi5.

6. Configuration applicative

Le fichier config.php est chargé par aggregate.php. S'il n'existe pas, le script lit config.sample.php. En production, il faut copier le sample vers config.php et adapter les paramètres.

Paramètre	Valeur type	Impact technique
trax_base_url	null ou http://127.0.0.1	Détermine l'URL cible utilisée par cURL pour joindre Trax réel.
aggregate_store_strategy	client_name	Si ILIAS perd le store, utilise le nom du client comme nom de store.
client_store_map	['ppm' => 'ppm']	Correspondance explicite client vers store, prioritaire sur la stratégie.
default_store	default	Store de repli si la stratégie n'est pas client_name.
max_statements_to_fetch	5000	Limite de statements lus pour construire le reporting ILIAS.
debug	false	Active ou masque les erreurs détaillées.

```

<?php
/**
 * Configuration de l'adaptateur Trax ILIAS Bridge.
 */
return [
    /**
     * null = utilise automatiquement le protocole et l'hôte de la requête entrante.
     * Exemple possible si Trax est joignable localement : 'http://127.0.0.1'
     *
     * L'adaptateur ajoute l'en-tête X-Trax-Ilias-Bridge-Bypass pour éviter les
     * boucles de réécriture Apache quand cette URL pointe vers le même vhost.
     */
    'trax_base_url' => null,

    /**
     * ILIAS 10 construit souvent l'URL d'agrégation ainsi :
     * /trax/api/gateway/clients/{client}/stores/api/statements/aggregate
     * Le nom du store est donc perdu.
     *
     * Stratégie par défaut : client_name
     * Exemple : client ppm -> store ppm.
     *
     * Pour les exceptions, utiliser client_store_map.
     */
    'aggregate_store_strategy' => 'client_name',

```

Paramètre	Valeur type	Impact technique
<pre> /** * Correspondance client Trax -> store Trax. * Exemple : * 'ppm' => 'ppm', * 'eform' => 'default', */ 'client_store_map' => [// 'ppm' => 'ppm', // 'eform' => 'default',], /** * Utilisé seulement si aggregate_store_strategy n'est pas client_name * et si le client n'est pas déclaré dans client_store_map. */ 'default_store' => 'default', /** * Nombre maximum de statements xAPI chargés depuis Trax pour construire * la réponse attendue par ILIAS Learning Experiences / Ranking. */ 'max_statements_to_fetch' => 5000, /** * true = erreurs détaillées en JSON. * Laisser false en production. */ 'debug' => false,]; </pre>		

Choix de trax_base_url

- null : l'adaptateur reconstruit automatiquement le protocole et l'hôte de la requête entrante.
- URL forcée : utile si Trax est accessible par une URL interne, un autre vhost, un port local ou un backend qui évite une ré-interception Apache.
- En cas de boucle ou d'erreur 500 répétitive lors des appels sortants, forcer trax_base_url vers l'URL interne réelle de Trax et vérifier les logs Apache.

7. Flux HTTP

7.1 Flux cmi5 tokens

Étape	Détail technique
1	ILIAS appelle POST/GET /trax/api/gateway/clients/{client}/stores/{store}/cmi5/tokens.
2	Apache AliasMatch envoie la requête vers aggregate.php.
3	aggregate.php supprime activity_id de \$_GET et du corps si présent.
4	Le script reconstruit l'URL cible Trax avec activityId conservé.
5	cURL transmet la requête à Trax en conservant Authorization, Accept, Content-Type et X-Experience-API-Version.
6	La réponse Trax est relayée à ILIAS avec le statut et le corps d'origine.

7.2 Flux aggregate

Étape	Détail technique
1	ILIAS appelle GET /api/statements/aggregate?pipeline=...
2	L'adaptateur résout client et store.
3	L'adaptateur appelle Trax /xapi/statements?limit={max_statements_to_fetch}.
4	Le payload JSON Trax est décodé et la liste statements est extraite.
5	La pipeline ILIAS est décodée puis appliquée en PHP.
6	La réponse JSON compatible ILIAS est renvoyée.

7.3 Flux xAPI non-cmi5

Aucun flux d'écriture xAPI standard ne doit passer par aggregate.php. Un POST vers /xapi/statements doit être pris en charge par Trax directement.

8. Authentification et en-têtes

L'adaptateur ne possède pas d'identifiants Trax en propre. Il relaie l'Authorization fournie par ILIAS afin que Trax applique ses contrôles habituels sur le client et le store.

En-tête	cmi5/tokens	aggregate	Commentaire
Authorization	Relayé	Relayé	Nécessaire pour Basic Auth Trax.
Accept	Relayé	Relayé	Conserve le format accepté par ILIAS/Trax.
Content-Type	Relayé	Non utilisé	Utile pour nettoyer un corps JSON ou form-urlencoded.
X-Experience-API-Version	Relayé	Forcé à 1.0.3	Évite les doublons sur aggregate.
X-Trax-IlIAS-Bridge-Version	Ajouté	Ajouté	Traçabilité de version.
X-Trax-IlIAS-Bridge-Bypass	Ajouté	Ajouté	Marqueur technique pour les requêtes sortantes.
SetEnvIf Authorization "(.*)" HTTP_AUTHORIZATION=\$1			

9. Tests techniques

9.1 Test xAPI about

```
curl -i -u ppm:***** -H "X-Experience-API-Version: 1.0.3"
"http://SERVEUR_TRAX/trax/api/gateway/clients/ppm/stores/ppm/xapi/about"
```

9.2 Test aggregate verbes

```
curl -i -u ppm:*****
"http://SERVEUR_TRAX/trax/api/gateway/clients/ppm/stores/api/statements/aggregate?pipeline=%5B%7B%22%24group%22%3A%7B%22_id%22%3A%22%24statement.verb.id%22%7D%7D%5D"
```

9.3 Test cmi5 tokens

Le test le plus représentatif est le lancement depuis ILIAS, car ILIAS construit les paramètres réels attendus par le contenu cmi5. L'absence d'erreur activity_id et la présence de statements complets dans Trax confirment la correction.

9.4 Test non-régression xAPI non-cmi5

```
tail -f /var/log/httpd/access_log | grep -E "xapi/statements|cmi5/tokens|statements/aggregate"

# Attendu pour xAPI non-cmi5 :
POST /trax/api/gateway/clients/ppm/stores/ppm/xapi/statements
```

10. Supervision et logs

- Logs Apache access_log : vérifier les routes réellement appelées.
- Logs Apache error_log : détecter erreurs PHP, permissions, boucle, timeouts cURL.
- Logs ILIAS : rechercher LRS error, cmi5, xapi, exception, account, activity_id.
- Trax : vérifier la présence des statements et la réponse de /xapi/about.

```
grep -iE "cmix|ranking|account|LRS error|exception|error|500" /var/www/logs/ilias.log
journalctl -u httpd -n 100 --no-pager
tail -f /var/log/httpd/error_log
```

11. Performance

L'endpoint aggregate lit un nombre limité de statements via /xapi/statements, puis applique la pipeline en mémoire PHP. Le paramètre max_statements_to_fetch est donc important. Une valeur trop basse peut tronquer le reporting ; une valeur trop haute augmente la mémoire et le temps de réponse.

Paramètre	Effet si trop bas	Effet si trop haut	Recommandation
max_statements_to_fetch	Learning Experiences ou Ranking incomplet.	Temps de réponse plus long, consommation mémoire plus élevée.	Conserver 5000 au départ, ajuster selon volumétrie et monitoring.

12. Sécurité

- Utiliser HTTPS entre ILIAS et Trax en production.
- Limiter les droits fichiers : config.php en 0640, propriétaire apache.
- Ne jamais commiter config.php si l'environnement contient des valeurs sensibles.
- Laisser debug à false en production.
- Restreindre les accès réseau au serveur Trax selon l'architecture.
- Surveiller les accès aux routes cmi5/tokens et statements/aggregate.

13. Dépannage

Symptôme	Cause probable	Diagnostic	Correction
HTTP 404 HTML sur aggregate	AliasMatch non chargé ou route non interceptée.	grep AliasMatch + access_log.	Installer la conf v2.0.3, apachectl configtest, restart httpd.
Erreur activity_id	Route cmi5/tokens non interceptée.	grep cmi5/tokens dans conf et logs.	Corriger la conf Apache et redémarrer.
Erreur 401/403	Authorization absente ou credentials erronés.	Tester curl -u et SetEnvIf Authorization.	Corriger identifiants ILIAS ou Apache.
Ranking vide	Statements sans score ou acteur exploitable.	Lire result.score et actor.account.	Corriger le contenu ou accepter la limite fonctionnelle.
Learning Experiences vide	Mauvais object.id ou store mal résolu.	Comparer Activity ID ILIAS et statements Trax.	Corriger ILIAS ou client_store_map.
xAPI non-cmi5 perturbé	Règle Apache trop large.	grep AliasMatch RewriteRule.	Supprimer toute règle capturant /xapi/statements.
Timeout aggregate	Volumétrie élevée ou Trax lent.	Mesurer temps curl /xapi/statements.	Ajuster limit, indexation, volumétrie ou architecture.

14. Upgrade et rollback

Mise à jour

1. Sauvegarder aggregate.php, config.php et la conf Apache en place.
2. Déployer le nouveau aggregate.php et le nouveau fichier Apache si nécessaire.
3. Comparer config.sample.php avec config.php existant.
4. Exécuter apachectl configtest.
5. Redémarrer Apache.
6. Rejouer les tests cmi5, aggregate, Ranking et xAPI non-cmi5.

Rollback

```
cp /var/backups/trax-iliad-adapter/aggregate.php /var/www/trax-iliad-adapter/aggregate.php
cp /var/backups/trax-iliad-adapter/trax-iliad-adapter.conf /etc/httpd/conf.d/trax-iliad-adapter-
adapter.conf
apachectl configtest
systemctl restart httpd
```

15. Annexes techniques

Historique fonctionnel des corrections

Version	Domaine	Correction
2.0.1	Reporting	Évite le doublon d'en-tête X-Experience-API-Version sur les appels statements/aggregate.
2.0.2	Ranking ILIAS	Ajoute l'évaluation des accumulateurs nécessaires au classement : \$last, \$first, \$push, \$max, \$min, \$sum.
2.0.3	Apache	Remplace la logique RewriteRule par AliasMatch pour forcer l'interception avant Laravel sur les installations concernées.